Client-Side Scripting II

MCAST

Presented by: Carlo Mamo

Bachelors of Science (Hons) (MQF Level 6)

Chapter 3 – Routing and Http Requests Using Axios

Linting

- The two frequently used tools for maintaining the code standards in the Vue.js projects are **ESLint** and **Prettier**.
- **Linting** is an automated process for static analysis of the codebase for potential errors and inconsistencies with the project's coding standards.
- Usually, linter detects errors such as *missed commas, unused variables, unused imports, etc*. Linter cannot save us from errors related to the business logic; however, it will ensure that our code is syntactically correct and follows the best practices.
- The most popular linting tool for Javascript is **ESLint**.

Difference between formatting and linting

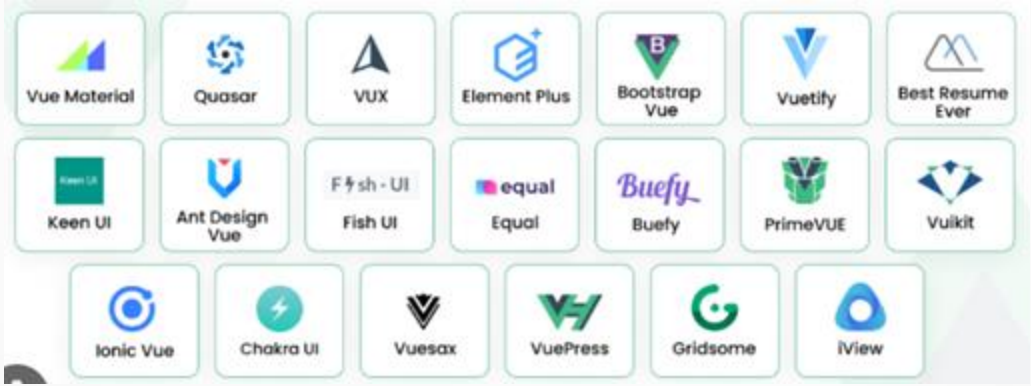
- The linter is more focused on detecting the errors while the formatter takes care of maintaining one standard in the codebase and follows several rules like **tab width, single/double quotes, and so on.**

- To have more control over the formatting standards in the project, we should use a separate tool called **Prettier**.

- Prettier is a more complete and robust formatting tool oriented only on formatting. Therefore Prettier and ESLint are a perfect combination in keeping the coding standards in the codebase.

UI component libraries and frameworks

- Apart from the CSS Framework **Bootstrap o**ther UI component libraries and frameworks that can be used with VUE include **Vuetify** and **Quasar** which follow the Material Design guidelines, **Buefy, Element Plus, PrimeVUE** and others.



Single Page Applications

- Single Page Applications are web apps or sites that interact with the user by dynamically rewriting the current page rather than loading entire new pages from the server.
- This approach allows us to only fetch the data/section of our page that is needed when a user interacts with our app.
- By dynamically rewriting smaller chunks of our site, it prevents us from re-downloading already loaded resources such as the images, scripts, CSS, etc.

Single Page Applications

- As a result SPA's tend to improve the user experience by:

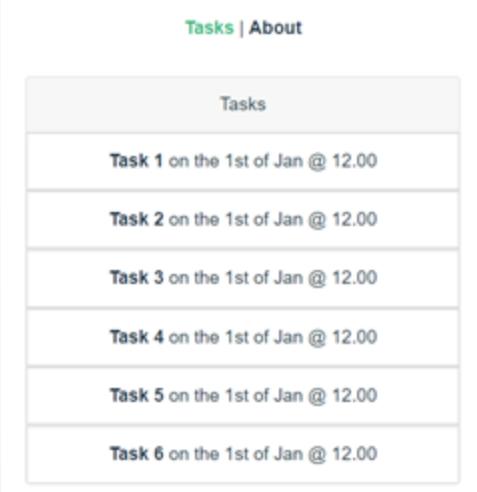
1. Providing faster load times between page navigations
2. Behaving more like traditional desktop applications

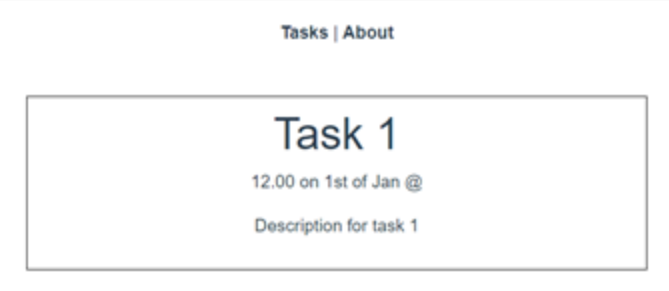
Vue Router

- Vue Router is the official router for Vue.js
- We need to use the Router so that we can navigate between the different pages of our app
- Feature of Vue Router include: Nested routes, dynamic routing, route params and query, and more.

Routing

- We will now create a Todo List. This app will show a list of tasks sorted by date which are retrieved through an API call and when you click on one task further details about that specific task will be shown.

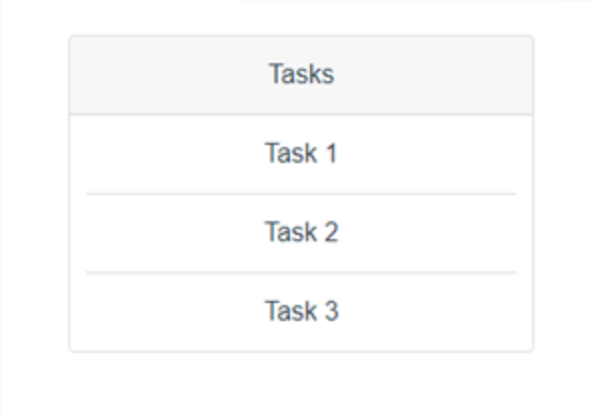




Exercise 1 - Display data of one object

- Create a new Vue application using **npm create vue@latest**. Select Routing when creating the new project. Replace all template code in HelloWorld.vue with the following and rename the file to TaskCard.vue:

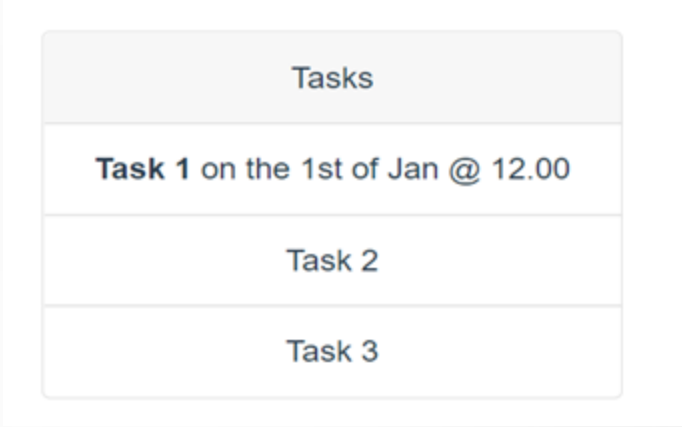
```
<div class="card" style="width: 18rem;">
  <div class="card-header">
    Tasks
  </div>
  <ul class="list-group list-group-flush">
    <li class="list-group-item">Task 1</li>
    <li class="list-group-item">Task 2</li>
    <li class="list-group-item">Task 3</li>
  </ul>
</div>
```



Exercise 1 – Display data of one object

- Delete all style and display list in the center of the page. Add one task object as a ref having the below properties and values and display as shown below. To access items in task object use ***{{ task.title }}*** *etc. Use the TaskCard component in App.vue*

```
id: 1,
title: "Task 1",
description: "Description for task 1",
location: "Valletta",
date: "1st of Jan",
time: "12.00",
```



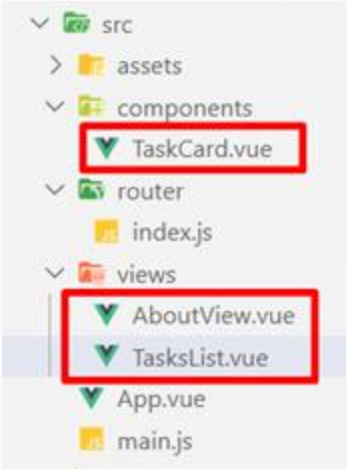
Exercise 2 – Displaying list from array

- Delete or comment out the task object from TaskCard.vue. TasksList.vue needs to have the collection (array) of tasks so that it can create a task card component for each task.

TaskCard.vue

```
<template>
  <div class="card" style="width: 18rem">
    <!-- <div class="card-header">Tasks</div> -->
    <ul class="list-group list-group-flush">
      <li class="list-group-item">
        <b>{{ task.title }}</b> on the {{ task.date }} @ {{ task.time }}
      </li>
      <!-- <li class="list-group-item">Task 2</li>
      <li class="list-group-item">Task 3</li> -->
    </ul>
  </div>
</template>

<script setup>
const props = defineProps({ task: Object });
```



TasksList.vue

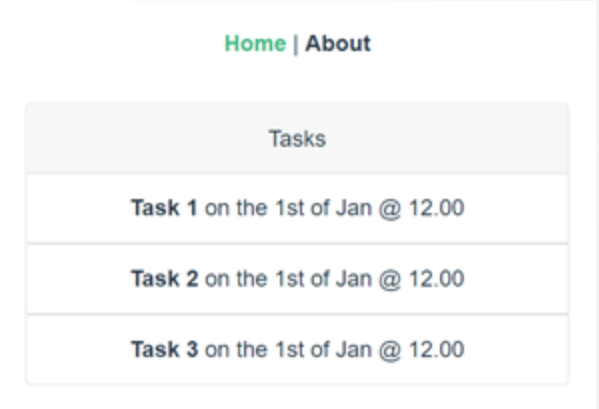
```
const tasks = ref([
  {
    id: 1,
    title: "Task 1",
    description: "Description for task 1",
    location: "Valletta",
    date: "1st of Jan",
    time: "12.00",
  },
  {
    id: 2,
    title: "Task 2",
    description: "Description for task 2",
    location: "Valletta",
    date: "1st of Jan",
    time: "12.00",
  },
  {
    id: 3,
    title: "Task 3",
    description: "Description for task 3",
    location: "Valletta",
    date: "1st of Jan",
    time: "12.00",
  },
]);
```

Exercise 2 – Displaying list from array

- TaksList.vue needs to pass each task to the child through a prop.
- Add for loop to create and display all tasks.

TasksList.vue

```
<div class="card" style="width: 24rem;">
  <div class="card-header">
      Tasks
  </div>
  <TaskCard v-for="task in tasks" :key="task.id" :task="task"/>
  </div>
```



Routing – Client-side vs Server-side

- **Server-side routing** – User types in a url or clicks a link and a request is sent to the server. The server will return the page. Refresh occurs on every page load.
- **Client-side routing** – When a user types in a url a request is sent to the server which in turn returns the index.html page and includes the rest of the app. The app has all the code that it needs and with the help of Vue Router it only renders the differences WITHOUT having to reload the page.

main.js

```
import "./assets/main.css";
import "bootstrap";
import "bootstrap/dist/css/bootstrap.min.css";

import { createApp } from "vue";
import App from "./App.vue";
import router from "./router";

createApp(App).use(router).mount("#app");
```

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: "/",
      name: "TasksList",
      component: TasksList,
    },
    {
      path: "/about",
      name: "about",
      component: About,
    },
  ],
});
```

App.vue > ...

```
<template>
  <div>
    <nav>
      <router-link to="/">Home</router-link> |
      <router-link to="/about">About</router-link>
    </nav>
    <router-view />
  </div>
</template>
```

Next Steps

- Change the about page and specify the component being used in the index.html (router folder)

```
import { createRouter, createWebHistory } from 'vue-router'
import TasksList from '../views/TasksList.vue'
import About from '../views/About.vue'

const routes = [
  {
    path: '/',
    name: 'TasksList',
    component: TasksList
  },
  {
    path: '/about',
    name: 'About',
    component: About
  }
```

API calls with Axios

- Currently tasks are hard-coded within the data of our TasksList.vue component. However in a real application we want **to fetch this data using an API call**.
- Hence we will make a request to our server for the tasks and the server will send back a JSON response with those tasks which will be then displayed on screen.
- Till now we have used the fetch() to make HTTP requests. In this exercise we will make use of the tool **Axios to make the API calls**.

Axios

- Very popular 3rd party JavaScript library to make API calls
- Response data is available as a JSON by default hence can be immediately be accessed using the data property.
- Promise-based HTTP client
- Add authentication to each request
- Set timeouts if requests take too long
- Intercept requests to create middleware
- Handle errors and cancel requests
- Downside – you have to add an extra package

```
axios.get('https://codilime.com/')
  .then(response => console.log(response.data));
```

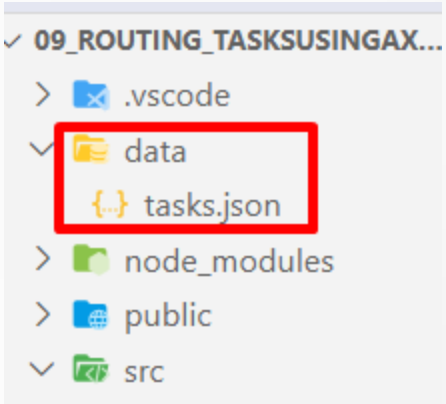
Native fetch() API

- Fetch like Axios is a promise-based HTTP-client.
- Fetch API provides the fetch() method and can fully reproduce all of the core features of Axios.
- Fetch API is a native interface that can be a bit harder to use.
- For more info on both options: Link

```
fetch('https://codilime.com/')
  .then(response => response.json())
  .then(console.log);
```

Adding JSON Server

- We will use JSON Server to be able and work with local data that simulates how an API behaves.
- Create a **mock database** to pull data from by creating a tasks.json file in a data folder as shown below. Make sure the data folder is in the top root.
- Install the local server package: **npm i json-server@0.17.4**



Adding JSON Server

- Add the following to package.json

```
"scripts": {
  "dev": "vite",
  "server": "json-server --watch data/tasks.json",
  "build": "vite build",
  "preview": "vite preview"
},
```

- Watch the file we created (tasks.json). In the terminal type **npm run server**:
- Check the following in the browser: **localhost:3000/tasks**

Adding Axios

- **npm i axios**
- Set the tasks array to empty.
- *import axios from 'axios'* in TasksList.vue
- Inside <script setup> or lifecycle hook *onMounted()* make the API call.

```
onMounted(() => {
  axios
    .get("http://localhost:3000/tasks")
    .then((response) => {
      tasks.value = response.data;
    })
    .catch((error) => {
      console.log("ERRRORS" + error);
    });
});
```

Creating a dedicated location for API calls

- As things stand Axios is being imported in the TasksList component but later we will need to import it again when retrieving task individual data.
- Importing Axios for each component which will need to make an API call becomes a bit problematic since each component will create a new Axios instance.
- To fix the above and make it better organized and simpler we will create a dedicated location to handle all API calls by creating a **TasksService.js**

TasksService.js

- Create a new folder services under the src directory.
- Inside this folder create the TasksService.js and refactor code in TasksList.vue to use the getTasks() from the TasksService.js

TasksService.js | TasksList.vue

src > services > TasksService.js > ...

```
1  import axios from "axios";
2
3  const apiClient = axios.create({
4    baseURL: "http://localhost:3000",
5    withCredentials: false,
6    headers: {
7      Accept: "application/json",
8      "Content-type": "application/json",
9    },
10 });
11
12 export default {
13   getTasks() {
14     return apiClient.get("/tasks");
15   },
16 };
```

```
onMounted(() => {
  TasksService.getTasks()
    .then((response) => {
      tasks.value = response.data;
    })
    .catch((error) => {
      console.log("ERRRORS" + error);
    });
});
```

Exercise 3

- Add JSON Server and create tasks.json
- Install Axios
- Populate the tasks array using Axios from localhost:3000/tasks
- Refactor API call into a Service Layer

Firebase Realtime Db vs Cloud Firestore

- Firebase offers 2 cloud-based, client-accessible database solutions.
- Cloud Firestore is Firebase's newest database for mobile app development. It builds on the successes of the Realtime Database with a new more intuitive data model. It also has faster queries and scales further than the Realtime Database.
- Realtime Database is Firebase's original database. It's an efficient, low-latency solution for mobile apps that require synced states across clients in real time.
- To check which one is suitable for your app check this link: https://firebase.google.com/docs/database/rtdb-vs-firestore

References

Vuejs.org. 2021. *Vue.js*. [online] Available at: <https://vuejs.org/> [Accessed 7 February 2021].

Schweiger, P., 2021. *Vue Mastery*. [online] Vue Mastery. Available at: <https://www.vuemastery.com/> [Accessed 7 February 2021].

Vue School. 2021. *Learn Vue.js from core-team members and industry experts at Vue School*. [online] Available at: <https://vueschool.io/> [Accessed 7 February 2021].

Udemy. 2021. *Develop with VueJS 2 (Complete Vue.js Router and Vuex Course)*. [online] Available at: <https://www.udemy.com/course/vuejs-2-the-complete-guide/> [Accessed 7 February 2021].

Any questions?

THANK YOU

Email: carlo.mamo@mcast.edu.mt